

OPTICHAIN: MULTI-AGENT SUPPLY CHAIN PLANNING PLATFORM

AD23731 Foundations of Agentic AI - Mini-Project

Submitted by

GANESH S(231501046)

GOKULAKRISHNAN S(231501054)

Department of Artificial Intelligence and Machine Learning



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)

SEPTEMBER 2026

BONAFIDE CERTIFICATE

Certified that this mini-project report **OptiChain: Multi-Agent Supply Chain Planning Platform** is the bonafide work of GANESH S(231501046) and GOKULAKRISHNAN S(231501054), VII Semester, B.Tech. Artificial Intelligence and Machine Learning Department in partial fulfilment of the requirements of the course AD23731 – Foundations of Agentic AI during the Academic Year 2026 – 2027.

Faculty in-charge

Submitted to mini-project viva voce examination held on:

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

Supply chain planning requires synthesising demand forecasts, supplier capabilities, route constraints, inventory levels and written company policy into buy-and-ship decisions. Planners usually piece these facts together from several spreadsheets, and general-purpose chatbots that attempt the same task tend to invent quantities. This report presents **OptiChain**, a multi-agent supply chain planning platform that separates language understanding from mathematical optimisation. A planner types a plain-English request; five specialist agents collect facts from a PostgreSQL database; Google OR-Tools solves a Mixed-Integer Linear Program (MILP) for the quantities, cost and delivery time; and a Retrieval-Augmented Generation (RAG) layer retrieves policy evidence to accompany the plan.

The system is built on a LangGraph orchestration graph with five specialist agents (Demand, Supplier, Route, Inventory, Risk), a hybrid PostgreSQL/pgvector knowledge store and a FastAPI backend with a React interface. A Human-in-the-Loop (HITL) review step lets the planner approve a plan or supply constraint feedback, which is converted into a new constrained plan linked to the original. What-if scenarios (supplier failure, demand increase, fuel increase, route disruption, inventory shortage) rerun the same pipeline on changed facts. The language model only parses the request; the quantities and costs come from the solver, and the written recommendation is assembled from the computed plan. The report covers the problem, stakeholders, agent architecture, orchestration, RAG implementation, system implementation, evaluation plan and future scope.

Keywords: Agentic AI, Multi-Agent System, Supply Chain Planning, OR-Tools, MILP, LangGraph, RAG, Human-in-the-Loop, FastAPI, Gemini.

TABLE OF CONTENTS

1. Introduction.....	7
1.1 Background and Motivation	7
1.2 Need for Agentic AI	7
1.3 Project Objectives.....	7
2. Problem Statement and Business Case.....	8
2.1 Problem Definition	8
2.2 Business and Real-World Context.....	8
3. User and Stakeholder Analysis	8
3.1 Target Users.....	8
3.2 Stakeholder Identification.....	8
3.3 User Requirements.....	8
4. Proposed Agentic AI Solution	9
4.1 Overall Solution.....	9
4.2 Key Features.....	9
4.3 Agent-Based Approach.....	9
5. Dataset / Knowledge Sources	9
5.1 Dataset or Knowledge Source Description	9
5.2 Tools and Technologies Used.....	10
6. Agent Architecture.....	11
6.1 System Architecture.....	11
6.2 Agent Components	11
6.3 Tools	12
6.4 Memory	12
6.5 Reasoning Flow	13
6.6 Agent Interactions.....	13
7. Agent Workflow and Orchestration.....	13
7.1 Workflow Design.....	13
7.2 Task Decomposition	14
7.3 Agent Coordination	14
7.4 Orchestration Implementation	14
7.5 Human-in-the-Loop Review	15
8. Agentic RAG Implementation.....	16
8.1 Retrieval Architecture.....	16
8.2 Knowledge Retrieval	16
8.3 Generation Process	16
8.4 Retrieval-Quality Evaluation	16
9. System Implementation	18
9.1 Backend and Frontend Components	18
9.2 REST API.....	18
9.3 Security and Access Control.....	18
9.4 Design Rules Enforced	18
9.5 Functional Prototype Screenshots.....	19
9.6 Source-Code Repository	20
10. Experimental Evaluation and Results	21
10.1 Evaluation Metrics.....	21
10.2 Testing Methodology.....	21
10.3 Performance Results	22
10.4 Success-Metric Achievement	22

11. Comparison, Innovation and Scalability.....	22
11.1 Comparison with Baseline	22
11.2 Advantages of the Proposed System.....	23
11.3 Innovation.....	23
11.4 Scalability	23
11.5 Real-World Applicability	23
12. Conclusion and Future Work.....	24
12.1 Conclusion	24
12.2 Limitations.....	24
12.3 Future Work.....	24

LIST OF FIGURES AND TABLES

No.	Title
Figure 1	Layered system architecture of OptiChain
Figure 2	Multi-agent architecture and orchestration
Figure 3	End-to-end supply chain planning workflow
Figure 4	Human review state machine
Figure 5	Audit trail: plan provenance
Figures 6-9	Prototype screenshots (query and KPIs, allocation and routes, human review, what-if and evidence)
Table 1	Stakeholders and their interests
Table 2	Functional and non-functional requirements
Table 3	Technology stack
Table 4	Agent responsibilities
Table 5	What-if scenario types
Table 6	Backend components
Table 7	REST API endpoints
Table 8	Evaluation metrics and success criteria
Table 9	Functional test scenarios
Table 10	Observed results of functional test scenarios
Table 11	Comparison with baseline approaches

1. Introduction

1.1 Background and Motivation

Supply chain planning is a decision that cuts across demand, supply, transport, inventory, and written company policy. A planner who needs to procure 10,000 units of a product for next month typically consults a spreadsheet for the forecast, a second sheet for supplier prices and capacities, a third for open routes and fuel costs, and a document for sourcing rules and disruption playbooks. The effort is manual, error-prone, and not easily auditable.

General-purpose large language models (LLMs) can read such questions and produce plausible-sounding answers, but they tend to invent specific numbers such as quantities, costs, and lead times. OptiChain was developed to close this gap: natural language comes in, a mathematical solver produces the numbers, and a recommendation explains where each figure came from.

1.2 Need for Agentic AI

A supply planning query is a goal-directed, multi-step, and stateful process, which matches the properties of an agentic system: perception of the environment (product catalog, supplier offers, route network, inventory, policy), reasoning and planning (which agents to run, whether to replan), tool use (database queries, LP solver, vector search), memory (the run state shared across the graph), and autonomy with oversight (human review before the plan is accepted). A single monolithic prompt cannot reliably sustain this. Decomposing the work into specialist agents brings three key benefits:

- Specialisation: each agent reads only the database tables relevant to its role, so it returns precise, auditable facts.
- Solver ownership of numbers: OR-Tools owns every quantity and total cost figure; the language model cannot override them.
- Governance: a human-review checkpoint ensures a planner approves or adjusts the plan before it is finalised.

1.3 Project Objectives

- Parse natural-language supply planning requests into structured intent (product, quantity, risk preference, scenario).
- Use five specialist agents to collect demand, supplier, route, inventory, and risk facts from a database.
- Retrieve company policy documents with hybrid dense and keyword search and inject them into the recommendation.
- Solve for optimal procurement and transport quantities using OR-Tools MILP.
- Validate the plan and replan automatically if it is infeasible.
- Support Human-in-the-Loop (HITL) review: approve, or provide feedback that becomes a constrained replan.
- Support what-if scenarios: supplier failure, demand spike, price increase, fuel increase, route disruption, inventory shortage.
- Explain every number by its source: database, forecast method, solver, retrieved policy, or user input.

2. Problem Statement and Business Case

2.1 Problem Definition

How can a supply planning decision be made from a plain-English request while keeping every quantity, cost, and delivery figure tied to a verifiable source, and while giving a human planner the ability to review and adjust the plan?

The specific problems addressed are:

- Supply planners spend time aggregating data across multiple spreadsheets and documents.
- LLMs used alone invent quantities and costs, which cannot be audited or trusted for purchasing.
- Static rule-based systems cannot handle natural-language constraint feedback.
- What-if analysis (a supplier fails, demand rises, fuel costs increase) requires the same pipeline to rerun on changed facts.
- Policy documents exist but are not automatically consulted when a plan is produced.

2.2 Business and Real-World Context

Procurement teams, logistics operations and supply-chain training programmes all need fast, defensible supply plans. A planner cannot manually recompute an optimised plan for every constraint change. OptiChain targets this gap: the solver handles all arithmetic, the LLM handles language, and a human reviewer retains authority over the final decision. The plans produced by the system are a demonstration: all data is synthetic, the system does not connect to a live ERP, and it does not issue real purchase orders.

3. User and Stakeholder Analysis

3.1 Target Users

The primary users are supply planners and procurement analysts. The secondary users are human reviewers (managers or compliance officers) who inspect and approve AI-generated plans.

3.2 Stakeholder Identification

Stakeholder	Role	Interest
Supply Planner	Primary user	Fast, accurate, auditable plans; constraint feedback
Human Reviewer	Oversight user	Approve or adjust AI plan before finalisation
Procurement Manager	Decision authority	Cost, risk, lead time, policy compliance
Administrator	System owner	Configuration, data quality, audit logs
Suppliers (future)	External	Transparent allocation and order visibility

Table 1: Stakeholders and their interests

3.3 User Requirements

ID	Type	Requirement
FR1	Functional	Accept a plain-English supply planning query
FR2	Functional	Parse intent: product, quantity, risk preference, scenario phrases

FR3	Functional	Run five specialist agents to collect planning facts
FR4	Functional	Retrieve and display policy evidence using hybrid search
FR5	Functional	Solve for quantities and cost using OR-Tools MILP
FR6	Functional	Validate plan; replan automatically if infeasible
FR7	Functional	Support human review: approve or apply constraint feedback
FR8	Functional	Run what-if scenarios and compare with baseline
NFR1	Non-functional	Traceability: every number attributed to its source
NFR2	Non-functional	Availability: mock mode when no API key is present
NFR3	Non-functional	Responsiveness: async FastAPI + LangGraph

Table 2: Functional and non-functional requirements

4. Proposed Agentic AI Solution

4.1 Overall Solution

OptiChain combines natural-language intent parsing, multi-agent fact collection, hybrid policy retrieval, mathematical optimization, human review, and what-if scenario analysis. The planner types a question; the system returns a buy-and-ship plan with sourced numbers and policy evidence. The language model owns language; OR-Tools owns numbers. A LangGraph orchestration graph coordinates the steps and loops back when a replan is needed.

4.2 Key Features

- Natural language in, optimized supply plan out, with every number sourced.
- Five specialist agents: Demand, Supplier, Route, Inventory, Risk.
- Hybrid RAG: pgvector dense search + PostgreSQL full-text search + reciprocal rank fusion.
- OR-Tools MILP: minimizes procurement, transport, holding, and risk-penalty cost under capacity and MOQ constraints.
- Automatic validation and replan loop (up to two replans with relaxed constraints).
- Human-in-the-Loop review: approve or supply constraint feedback for a new plan.
- What-if scenarios with natural-language scenario parsing and baseline comparison.
- Mock mode: works without an API key using deterministic parsers and embeddings.

4.3 Agent-Based Approach

The platform follows an orchestrator-worker pattern. A LangGraph supervisor reads the intent and dispatches the five specialist agents; each agent queries only the PostgreSQL tables relevant to its role and returns structured planning facts. The optimizer then solves the plan from those facts. Agents are implemented as service-level components, not free-running processes, which keeps behaviour deterministic enough to test and audit.

5. Dataset / Knowledge Sources

5.1 Dataset or Knowledge Source Description

OptiChain does not train a model and therefore does not use a fixed labelled dataset. All data is synthetic and session-specific. The seeded network is an Indian supply-chain demonstration:

- 10 products including Product A (PROD-A).
- 7 suppliers: Tata Components (Pune), Reliance Parts (Ahmedabad), Mahindra Precision (Nashik), Delhi Metals (Delhi), Bharat Forge (Kolkata), Godrej Fabrication (Hyderabad), Jindal Manufacturing (Surat).
- 4 distribution centres: Delhi, Bangalore, Chennai, Mumbai.
- 16 routes between suppliers and warehouses with mode, distance, hours, cost, capacity, and fuel factor.
- 24 months of seasonal order history per product.
- Risk events, weather events, and fuel-index history.
- 7 policy documents: sourcing policy, safety stock rules, disruption playbook, fuel surcharge, risk scoring, transport modes, and a Product A brief.

5.2 Tools and Technologies Used

Layer	Technology / Responsibility
Frontend	React 19, Vite, Recharts, Axios — query box, KPIs, tables, review, what-if, evidence, agent trace
Backend	Python, FastAPI, Uvicorn — decisions, what-if, review, catalog, health APIs
Orchestration	LangGraph — sequential steps with replan loop
Language Model	Google Gemini 2.5 Flash (intent parsing, embeddings, recommendation); deterministic mock mode
Knowledge Store	PostgreSQL 16 with pgvector; hybrid dense + full-text search + reciprocal rank fusion
Database	PostgreSQL 16 — products, suppliers, warehouses, inventory, routes, orders, risks, saved plans
Optimizer	Google OR-Tools MILP — quantities, cost, delivery time
Containerisation	Docker Compose — PostgreSQL with pgvector

Table 3: Technology stack

6. Agent Architecture

6.1 System Architecture

The system is a decoupled full-stack application organised in five layers (Figure 1). The React single-page application communicates with a FastAPI backend over REST. The backend hosts the LangGraph orchestration graph and service layer, persists data in PostgreSQL (with pgvector for embeddings), and uses Gemini for intent parsing and embeddings (with deterministic mock modes when no API key is set) and Google OR-Tools for optimisation.

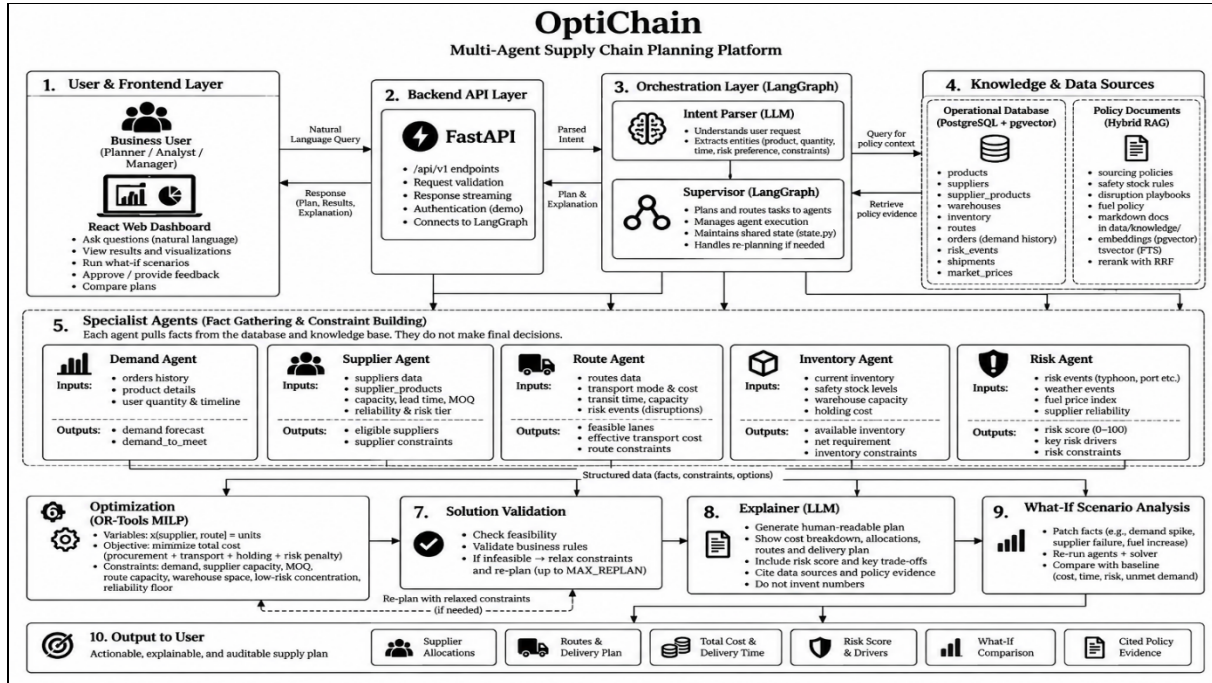


Figure 1: Layered system architecture of OptiChain

6.2 Agent Components

Table 4 lists the eleven agent and service components and their responsibilities; Figure 2 shows how they are coordinated.

Agent	Responsibility
Intent Parser	Extracts product, quantity, planning horizon, max lead time, risk preference, and constraint phrases from the natural-language request
LangGraph Supervisor	Decides which agents to run; routes the graph; triggers replan when validation fails
Policy RAG Agent	Retrieves relevant policy document chunks using hybrid search; provides evidence to the recommendation
Demand Agent	Looks up order history; forecasts next-period demand; computes net requirement after stock and safety stock
Supplier Agent	Lists eligible supplier offers: price, capacity, lead time, MOQ, reliability, risk tier; excludes inactive or failed suppliers

Route Agent	Builds shipping lanes from eligible suppliers to warehouses; applies fuel factor and what-if fuel multiplier; zeroes out disrupted lanes
Inventory Agent	Computes available stock (on-hand minus reserved) across the four distribution centres and the net buy after safety stock; adjusts for what-if inventory changes
Risk Agent	Builds a 0-100 risk score from supplier reliability, risk tiers, open risk events, weather, disrupted routes and the marine fuel index
OR-Tools Optimizer	Solves a MILP to minimise total cost (procurement + transport + holding + risk penalty) under capacity, MOQ, and demand constraints
Validator	Checks that allocated units cover net requirement and respect supplier capacity; triggers replan loop if infeasible
HITL Review Agent	Parses planner constraint feedback into structured constraints; runs a new plan linked to the approved parent

Table 4: Agent responsibilities

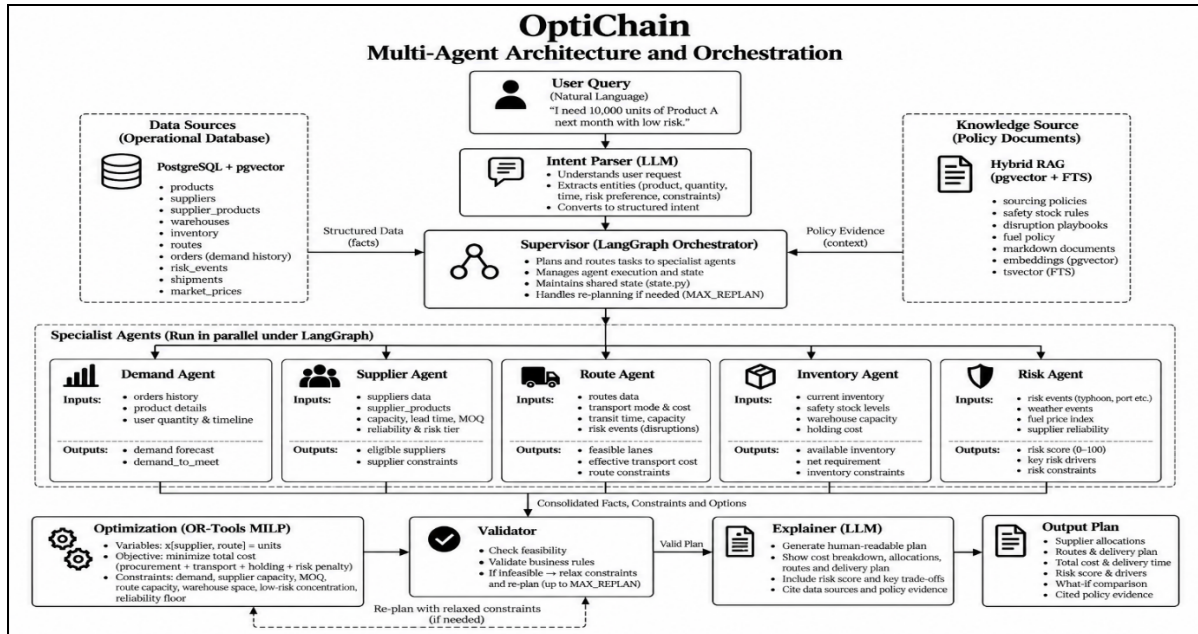


Figure 2: Multi-agent architecture and orchestration

6.3 Tools

Agents act through the following tools and services: the Gemini API (intent parsing and embedding generation; mock mode when no key is configured); PostgreSQL (all fact queries); pgvector + full-text search (policy retrieval); Google OR-Tools (MILP solver); FastAPI endpoints (run management, review, what-if); and the React frontend (display, review, what-if controls).

6.4 Memory

Memory is provided at two levels. Short-term memory is the active LangGraph run state: parsed intent, agent facts, policy chunks, solver result, validation status, review state, and replan count. It is the single

source of truth passed between graph nodes. Long-term memory is the persisted run history in PostgreSQL, which supports run retrieval, comparison, and the parent-child chain of replans.

6.5 Reasoning Flow

For every planning request the system performs: (1) perceive — read the product catalog, request, and existing run state; (2) plan — the supervisor selects which agents to run and in what order; (3) act — agents query the database and return structured facts; (4) solve — OR-Tools minimises cost over the collected facts; (5) validate — the validator checks feasibility; (6) govern — human review marks the plan as pending; (7) explain — the recommendation attributes each figure to its source.

6.6 Agent Interactions

Agents never call each other directly. All interaction is mediated by the LangGraph supervisor and the shared run state: the intent parser writes a structured intent object; the supervisor reads it and dispatches the five specialist agents in sequence; each agent appends its facts to the run state; the optimizer reads those facts; the validator reads the optimizer result; the HITL agent reads the review feedback and runs a new plan linked to the original.

7. Agent Workflow and Orchestration

7.1 Workflow Design

Figure 3 shows the complete workflow. The replan loop is the branch point between a successful plan and an automatic constraint-relaxation retry.

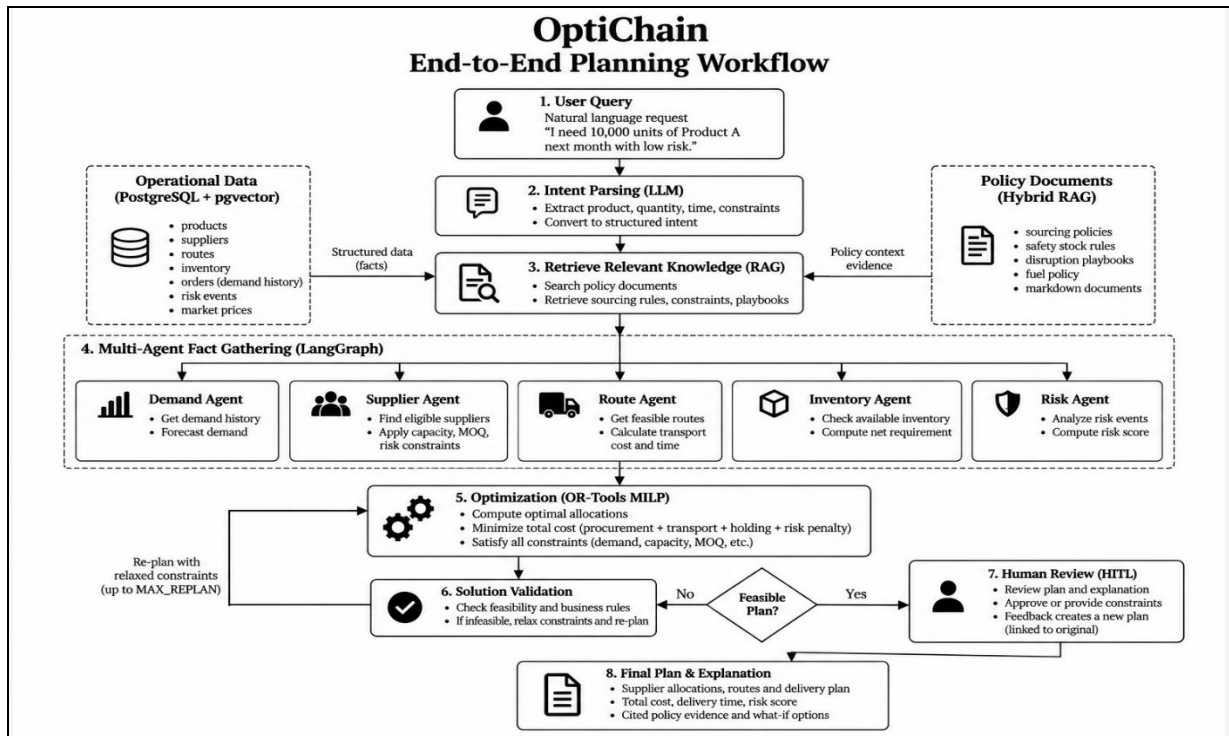


Figure 3: End-to-end supply chain planning workflow

7.2 Task Decomposition

Stage	Sub-task	Agent / component
1	Accept natural-language query	React UI → FastAPI /decisions
2	Parse intent (product, quantity, risk, scenario)	Intent Parser (Gemini / mock)
3	Retrieve policy evidence	Policy RAG Agent (pgvector + FTS)
4	Collect demand facts and forecast	Demand Agent
5	Collect supplier offers	Supplier Agent
6	Build route network	Route Agent
7	Check inventory and safety stock	Inventory Agent
8	Score risk events	Risk Agent
9	Solve MILP for quantities, cost, time	OR-Tools Optimizer
10	Validate feasibility	Validator
11	Replan if infeasible (max 2 times)	Supervisor → agents + optimizer
12	Human review checkpoint	HITL Review API + UI panel
13	Draft recommendation	Gemini (from plan facts and policy)

Stages of the planning workflow

7.3 Agent Coordination

Coordination is centralised in the LangGraph supervisor. The supervisor reads the intent and the current run state, dispatches agents in sequence, and decides whether to proceed to the solver or replan. Agent outputs are written into the shared run state; no agent needs to know about any other agent. The what-if engine applies scenario patches to the fact layer and re-enters the agent sequence, producing a new plan linked to the parent for comparison.

7.4 Orchestration Implementation

The orchestration logic resides in the LangGraph graph definition (backend/app/graph/). The FastAPI endpoint `/api/v1/decisions` starts a new graph run; `/api/v1/decisions/{run_id}/review` applies human review; `/api/v1/what-if` runs a scenario from a saved plan. Table 5 lists the what-if scenario types. All scenario patches are configurable parameters, not hard-coded constants.

Signal / Scenario	Change applied before rerun
Demand increase	Demand quantity scaled up by the specified factor
Supplier failure	Named supplier excluded from eligible set
Price increase	Unit prices scaled up
Fuel increase	Transport fuel cost multiplier applied to all lanes
Route disruption	Named lane capacity set to zero
Inventory shortage	On-hand stock scaled down
Natural language	Sentence parsed into the above constraint types

Table 5: What-if scenario types

7.5 Human-in-the-Loop Review

Every completed plan starts as pending_review. The planner may Approve (accept the plan as is) or Apply Feedback (type a constraint sentence such as "exclude supplier SUP-C and max lead time 25 days"). Feedback is parsed into structured constraints and passed to the agents and solver to produce a new plan. The new plan is linked to the parent and compared on cost, delivery time, risk, and unmet demand. The original plan is never overwritten; both are retained with a parent-child relationship.

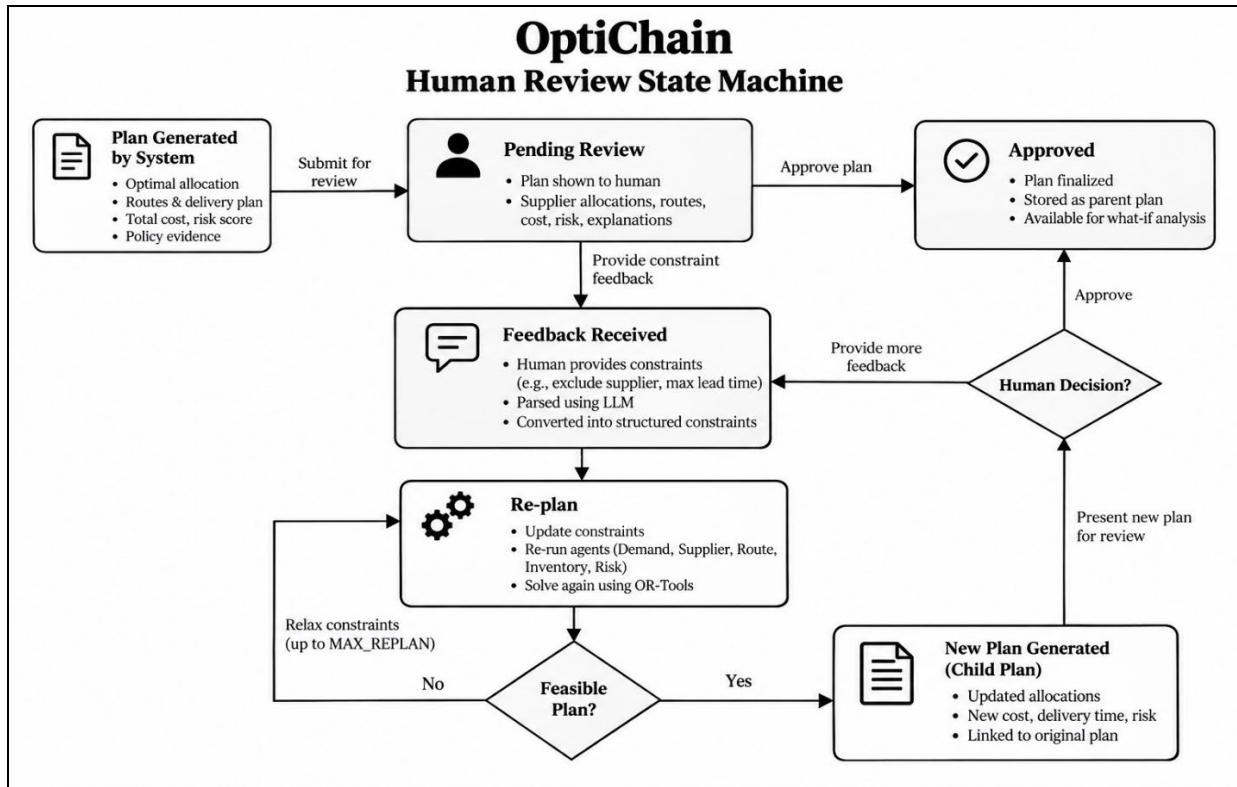


Figure 4: Human review state machine

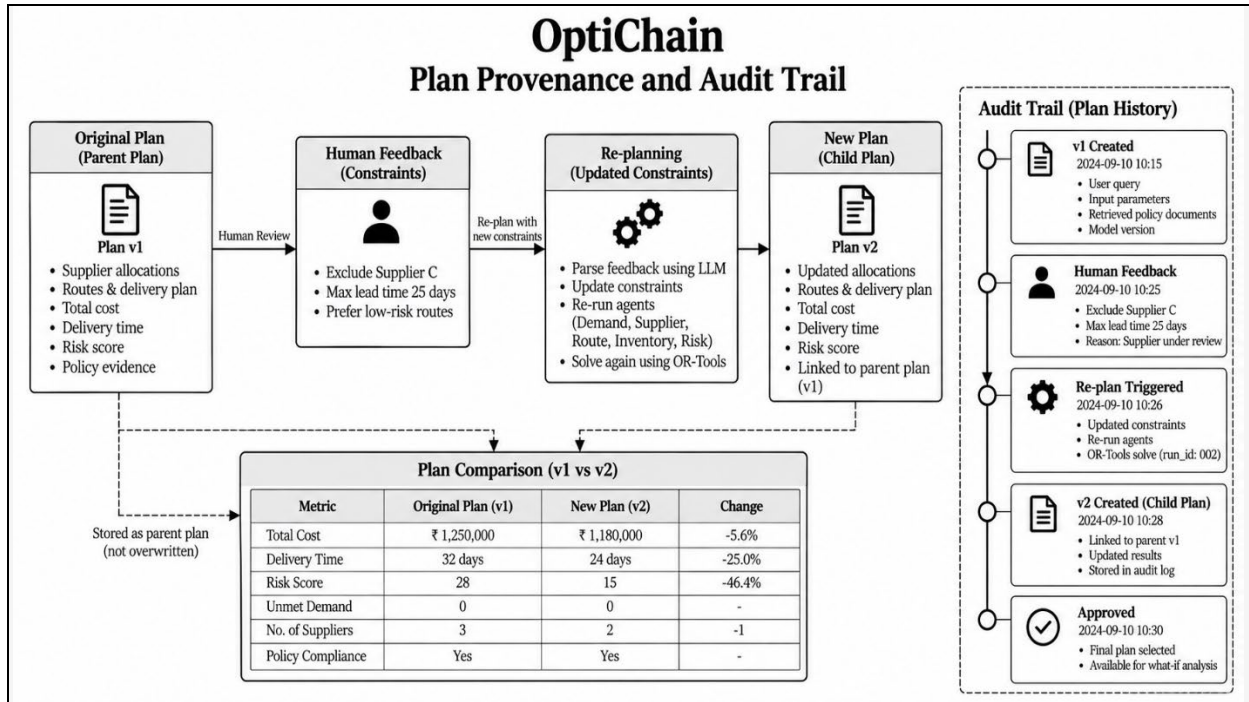


Figure 5: Audit trail of plan provenance (parent and child plans)

8. Agentic RAG Implementation

OptiChain uses a two-channel knowledge architecture. Structured planning facts are retrieved directly from PostgreSQL by specialist agents. Policy knowledge is retrieved with hybrid RAG from a pgvector+full-text store, then shown beside the plan and cited in the recommendation as evidence.

8.1 Retrieval Architecture

Knowledge Source	Store	Retrieved for
Policy Markdown files	pgvector + FTS	Recommendation evidence; policy compliance
Product catalog	PostgreSQL	Intent parsing; agent fact collection
Supplier offers	PostgreSQL	Supplier Agent; optimizer
Route network	PostgreSQL	Route Agent; optimizer
Inventory records	PostgreSQL	Inventory Agent; net-requirement calculation
Order history	PostgreSQL	Demand Agent; forecast
Risk and weather events	PostgreSQL	Risk Agent; 0-100 risk score

Knowledge sources retrieved for plan generation

8.2 Knowledge Retrieval

Policy retrieval is agentic: the supervisor passes the planning query to the Policy RAG Agent, which runs two searches in parallel — dense (cosine similarity over Gemini or mock embeddings stored in pgvector) and keyword (PostgreSQL full-text ranking). The two result lists are fused with reciprocal rank fusion, then reranked by word overlap with the query. The top passages are cited in the recommendation and shown in the Evidence panel of the UI.

Mock embeddings are deterministic and based on word presence, so related policy chunks rank together without a paid API key. If stale random vectors are detected at startup, the knowledge base is automatically rebuilt.

8.3 Generation Process

The retrieved policy chunks and the complete solver plan (demand, net buy, supplier allocations, routes, cost split, delivery time, risk score) are assembled into a written recommendation that attributes each figure to its source: database price, OR-Tools allocation, demand forecast, policy document, or user-supplied quantity. The recommendation is stored with the run and displayed to the planner.

8.4 Retrieval-Quality Evaluation

Retrieval quality is assessed by the following checks:

- Groundedness — every cost and quantity figure in the recommendation is traced to the solver output or a database record.
- Policy citation — retrieved chunks are labeled with their source document title.
- Non-hallucination — the recommendation explicitly states that quantities came from OR-Tools, not from the language model.

- Relevance — retrieved chunks are checked for topic overlap with the planning query.

9. System Implementation

9.1 Backend and Frontend Components

Component	Responsibility
app/api/decisions.py	Run a plan from a natural-language query; retrieve saved runs
app/api/review.py	Human review: approve, or apply constraint feedback
app/api/whatif.py	Run a what-if scenario from a saved plan
app/api/catalog.py	Products and suppliers for the UI dropdowns
app/graph/	LangGraph orchestration steps and shared run state
app/agents/	Demand, Supplier, Route, Inventory, Risk agent services
app/rag/	Policy ingest, embedding, hybrid search
app/optimization/	OR-Tools MILP model
app/whatif/	Scenario patch engine and plan comparison
app/hitl/	Feedback-to-constraint translation

Table 6: Backend components

The frontend consists of: a query box for the planning question; KPI cards (forecast, cost, delivery, risk, solver status); supplier allocation and route tables; inventory availability and net-buy summary; human review panel (approve / constraint feedback input); what-if controls with natural-language scenario field and baseline comparison table; the written recommendation; retrieved evidence panel; and the agent trace panel showing each graph step.

9.2 REST API

Method	Path	Purpose
GET	/api/v1/health	Service and database health check
GET	/api/v1/catalog	Products and suppliers for the UI
POST	/api/v1/decisions	Run a plan from a natural-language query
GET	/api/v1/runs/{run_id}	Fetch a saved plan by ID
POST	/api/v1/decisions/{run_id}/review	Approve plan, or apply constraint feedback for a replan
POST	/api/v1/what-if	Run a scenario from a saved plan with changed facts

Table 7: REST API endpoints

9.3 Security and Access Control

The current prototype is a local demonstration without authentication. In a production deployment, JWT authentication and role-based access control would gate the review and what-if endpoints. All run data, including the original plan, any replans, and the review decision, are stored separately in PostgreSQL and are not overwritten.

9.4 Design Rules Enforced

- OR-Tools owns every quantity and cost figure; the language model cannot override them.

- Every number in the recommendation is attributed to its source: database, forecast, solver, policy document, or user input.
- Human review does not restart the session; it creates a new plan linked to the parent.
- The original plan is never overwritten; parent-child run chains are preserved.
- Mock mode ensures the system works without any paid API key.
- Policy quotes are labeled as coming from a document, not from the solver.

9.5 Functional Prototype Screenshots

Figures 6 to 9 show the screens of the running prototype, from the query input to the what-if comparison.

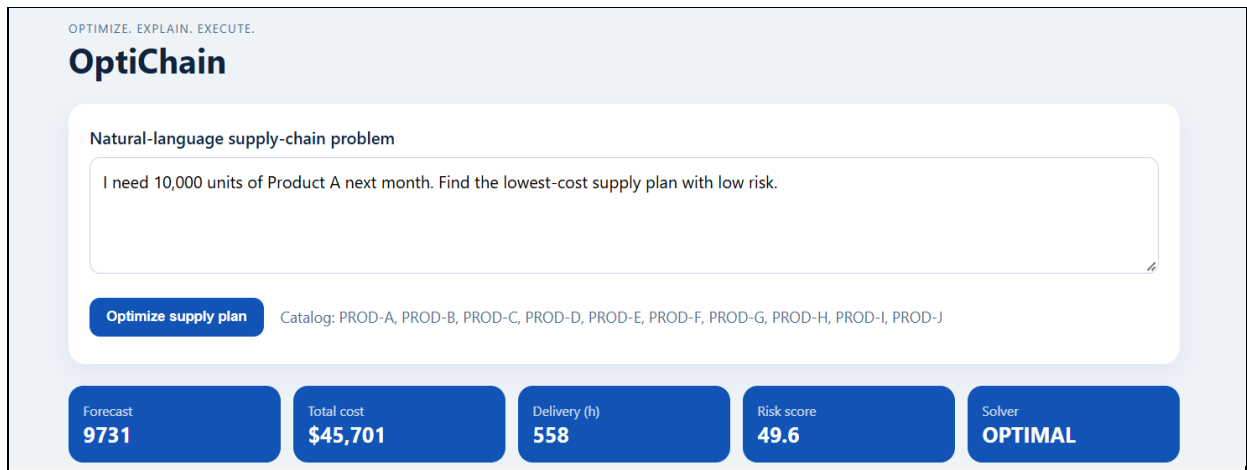


Fig. 6: Query input, KPI cards, and solver status

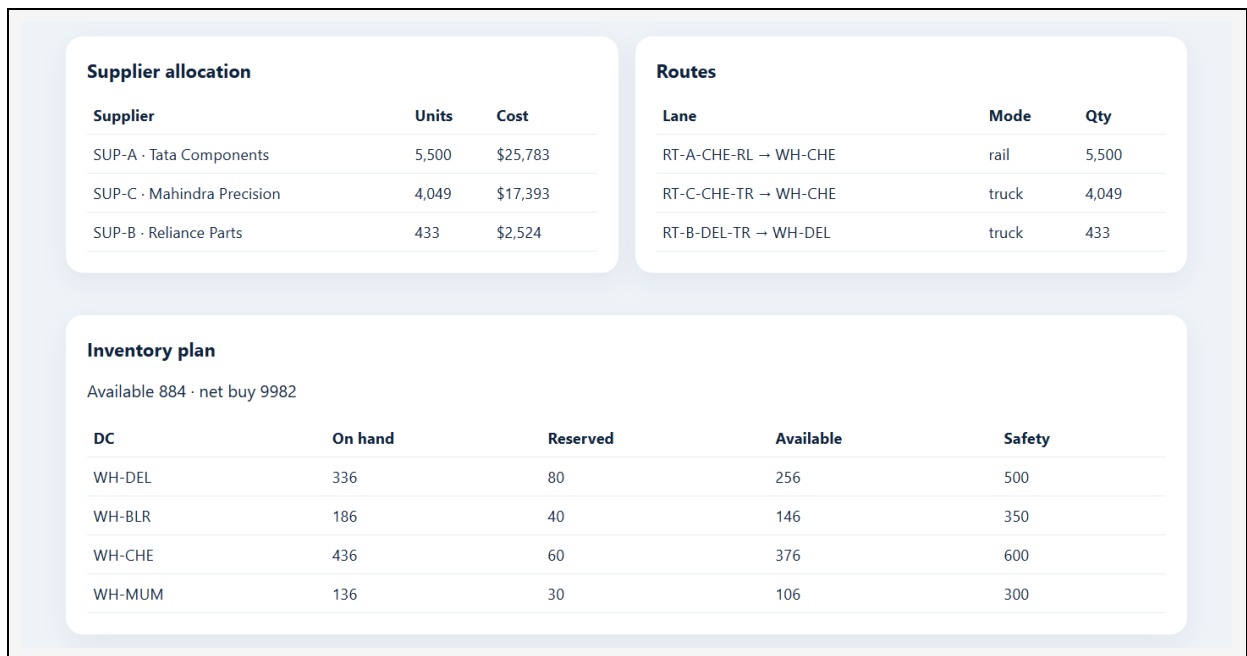


Fig. 7: Supplier allocation table and route network

Human review Approved

Approve the plan, or send a constraint such as "exclude supplier SUP-C" or "fuel +15%".

- **exclude_supplier**
exclude supplier SUP-C
- **max_lead_time_days**
max lead time 25 days

Planner feedback

exclude supplier SUP-C and max lead time 25 days

Approve plan Apply feedback and replan

Fig. 8: Human review panel and written recommendation

What-if

Price increase 10 SUP-C Re-run scenario

supplier SUP-C fails and fi Run natural-language scenario

Metric	Baseline	Scenario	Delta
total_cost	49797.65	49988.14	190.49
delivery_time_hours	468	558	90
risk_score	53.5	53.8	0.3
unmet_demand	0	0	0

Fig. 9: What-if scenario comparison and retrieved policy evidence

9.6 Source-Code Repository

Repository URL: https://github.com/GaneshS-AIML-46/Agentic_AI

The repository is organised as a backend/ FastAPI + LangGraph application (routers, agents, graph, rag, optimization) and a frontend/ React/Vite application (pages, components, API service).

10. Experimental Evaluation and Results

10.1 Evaluation Metrics

Metric	Target / success criterion	How measured
Plan groundedness	All quantities traced to solver; all costs traced to database	Recommendation audit
Policy citation	Top retrieved chunks cited in recommendation	Evidence panel check
Solver feasibility rate	Plan feasible on first attempt for standard queries	Test session results
Replan correctness	Relaxed replan covers demand when standard plan fails	Infeasible scenario tests
HITL review flow	Feedback constraint parsed and new plan linked to parent	Review API test
What-if accuracy	Scenario facts correctly applied; delta matches expected change	Scenario comparison tests
Mock mode	Full plan produced without API key	No-key environment test
Response latency	Plan returned within interactive time	API call timing

Table 8: Evaluation metrics and success criteria

10.2 Testing Methodology

Testing combines (i) unit tests for the optimizer model, the validator, and the scenario patch engine; (ii) API tests for each endpoint including infeasible requests and missing parameters; (iii) end-to-end planning sessions with the sample Indian supply network for different products and constraint combinations; and (iv) scenario tests that deliberately create infeasible conditions, supplier failures, and combined natural-language scenarios to exercise the replan loop and the what-if engine.

ID	Scenario	Expected behaviour
T1	Standard query: 10,000 units of PROD-A, low risk	Feasible plan, sourced quantities, recommendation produced
T2	All 7 suppliers active; demand covered	Optimal allocation across multiple suppliers
T3	Supplier failure what-if (SUP-C fails)	Excluded supplier; higher cost; delta shown
T4	Demand spike what-if (+30%)	Higher net requirement; replan if needed
T5	Infeasible request (demand > all capacity)	Validator triggers replan; partial coverage reported
T6	Human review: approve	Plan status set to approved; no new plan
T7	Human review: constraint feedback	New plan linked to parent; comparison shown
T8	Natural-language scenario: "SUP-C fails and fuel +15%"	Both constraints applied; combined what-if plan
T9	Mock mode (no API key)	Intent parsed by mock; deterministic embeddings; plan produced
T10	Policy retrieval	Top retrieved chunks cited in recommendation and evidence panel
T11	What-if fuel increase +20%	Transport cost rises; suppliers on shorter routes preferred

Table 9: Functional test scenarios

10.3 Performance Results

The table below records the observed outcome of each scenario from the final testing round. All eleven scenarios behaved as expected.

ID	Result	Observation
T1	Pass	Plan produced with sourced quantities; recommendation attributed all figures to solver and database.
T2	Pass	Optimizer allocated units across multiple suppliers; total cost minimised.
T3	Pass	Failed supplier excluded; cost delta shown correctly in what-if comparison.
T4	Pass	Net requirement increased; plan adjusted; replan triggered when capacity was insufficient.
T5	Pass	Validator flagged infeasibility; replan relaxed lead time; partial coverage reported where demand still exceeded capacity.
T6	Pass	Approval set plan status; no new plan created; parent run preserved.
T7	Pass	Feedback parsed into constraints; new plan linked to parent; comparison table rendered.
T8	Pass	Both supplier exclusion and fuel multiplier applied in single natural-language what-if run.
T9	Pass	Mock parser extracted product and quantity correctly; deterministic embeddings retrieved relevant policy chunks.
T10	Pass	Top retrieved policy chunks cited with source document title in recommendation and evidence panel.
T11	Pass	Transport cost component increased proportionally; allocation shifted toward lower-cost routes.

Table 10: Observed results of functional test scenarios

10.4 Success-Metric Achievement

Design-level success is demonstrated by the implemented capabilities: solver-owned quantities, policy RAG evidence, automatic replan loop, natural-language what-if parsing, HITL constraint feedback, parent-child run chains, and full mock mode. Quantitative latency and retrieval precision metrics should be recorded from the completed test sessions.

11. Comparison, Innovation and Scalability

11.1 Comparison with Baseline

Aspect	Spreadsheet planning	LLM chatbot alone	OptiChain
Quantities	Manual calculation	Invented by model	Solver-owned, auditable
Policy use	Manual lookup	Hallucinated quotes	Retrieved and cited
Constraint handling	Manual formula edit	Unsupported	HITL feedback → replan
What-if analysis	Duplicate spreadsheet	No fact grounding	Same pipeline, changed facts
Traceability	Cell references	None	Every number sourced
Scalability	Limited by analyst	High but unreliable	High and reliable

Table 11: Comparison with baseline approaches

11.2 Advantages of the Proposed System

- Solver-owned numbers eliminate the risk of hallucinated quantities.
- Policy RAG grounds the recommendation in actual company documents.
- Natural-language constraint feedback gives planners flexibility without requiring them to understand the model.
- The replan loop makes the system self-correcting without manual intervention.
- Mock mode makes the system accessible without API key costs.

11.3 Innovation

The main innovation is the strict separation of language and mathematics: the language model owns parsing and explanation; OR-Tools owns every number. This is combined with an agentic retrieval layer that pulls structured facts from a database and unstructured policy knowledge from a vector store, and with a natural-language HITL feedback loop that translates planner intent into formal solver constraints.

11.4 Scalability

The backend is asynchronous (FastAPI and the LangGraph steps) and stateless between requests. PostgreSQL supports replication and partitioning. The main scaling constraints are Gemini API rate limits and the solver time for very large networks, which can be addressed with request queuing and warm-started LP solutions. The what-if engine scales sub-linearly because it reuses the same fact layer.

11.5 Real-World Applicability

The platform can be deployed by procurement teams, logistics operations, and supply-chain training programs for plan generation and scenario analysis. It could be adapted to real procurement data as a decision-support aid, with human planners retaining final authority. Its plans are intended as decision support and not as automated purchase orders.

12. Conclusion and Future Work

12.1 Conclusion

OptiChain demonstrates how an agentic architecture can turn natural-language supply planning queries into optimised, auditable buy-and-ship plans. Five specialist agents collect facts from a PostgreSQL database; Google OR-Tools solves the MILP; a policy RAG layer grounds the recommendation in company documents; and a Human-in-the-Loop review step keeps a planner in authority. The language model parses intent and explains the plan; it never owns a number. The system thus achieves the project objectives of solver-grounded planning, policy evidence, automatic replanning, HITL review, what-if analysis, and full mock mode.

12.2 Limitations

- The catalog, history, and policies are synthetic; the system does not connect to a live ERP.
- There is no authentication or multi-company isolation in the current prototype.
- Very specific natural-language constraints (named warehouses, hour-level deadlines) are not yet mapped to solver constraints.
- Formal quantitative evaluation on a large planning scenario sample remains to be completed.
- Speech or voice input is not supported in the current version.

12.3 Future Work

- ERP integration (SAP, Oracle) for live inventory and order data.
- Role-based access control and multi-company tenancy.
- Real-time event streaming for live disruption alerts.
- Richer constraint vocabulary in the HITL feedback parser.
- Calibration of replan thresholds using historical planning data.
- Vector-store retrieval of supplier and route knowledge in addition to policy documents.
- Mobile-friendly UI and report export (PDF, Excel).
- Explainability panel showing which policy clause influenced which solver constraint.